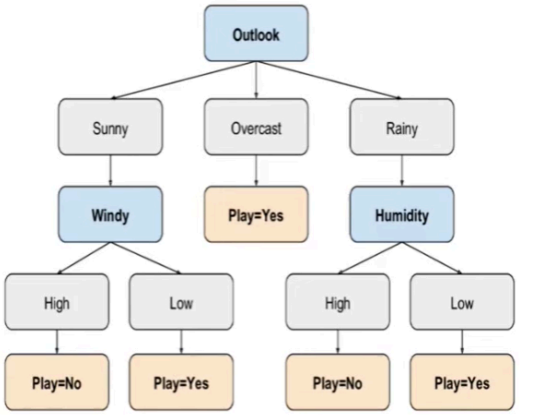
Decision Tree, its building process and Entropy

It looks like a Binary Tree. (**All the pictures collected from [Campusx]**)

1)

Day	Outlook	Temp	Humid	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Categorical Input Dataset



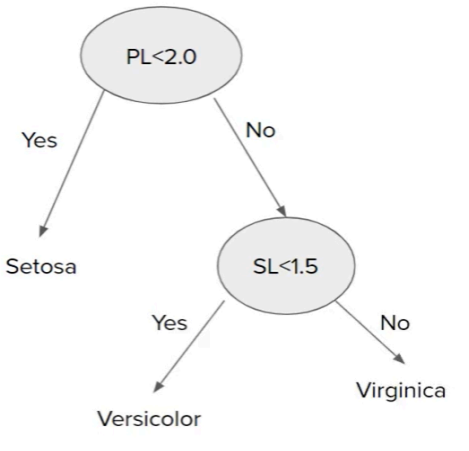
Decision Tree

2)

What if we have numerical data?

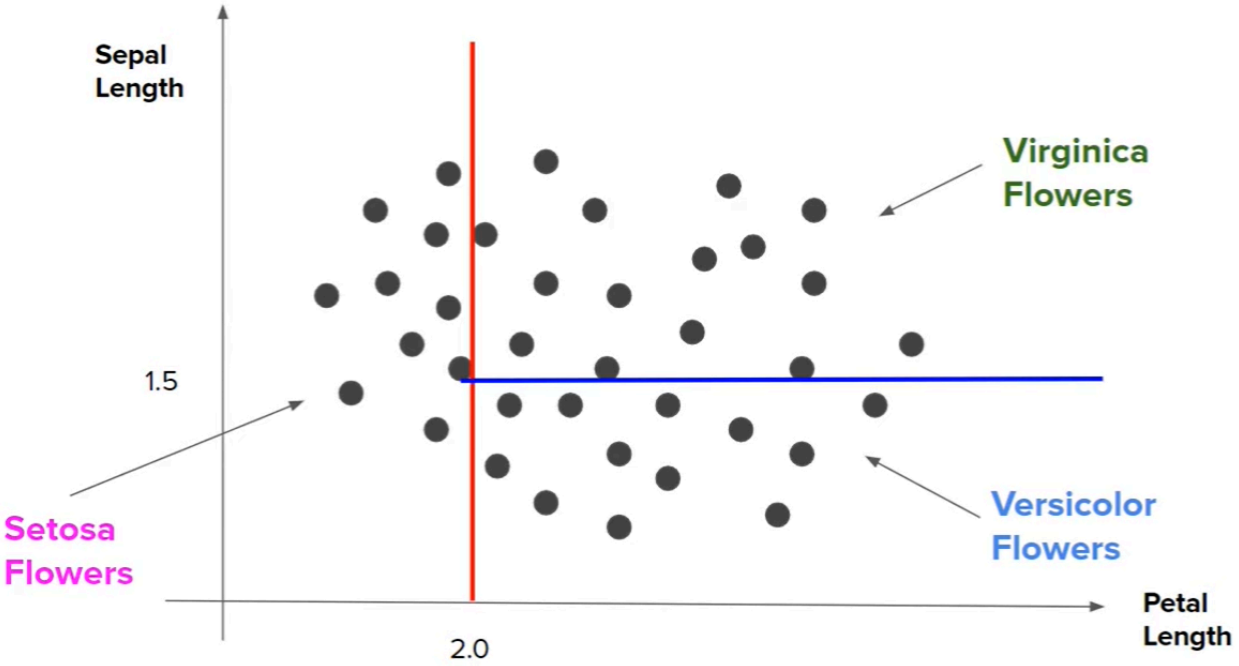
Petal Length	Sepal Length	Type
1.34	0.34	Setosa
3.45	1.45	Versicolor
1.69	0.98	Setosa
2.56	1.79	Virginica
3.00	1.13	Versicolor
1.3	0.88	Setosa

Numerical Input Dataset



Decision Tree

Geometric Intuition



3)

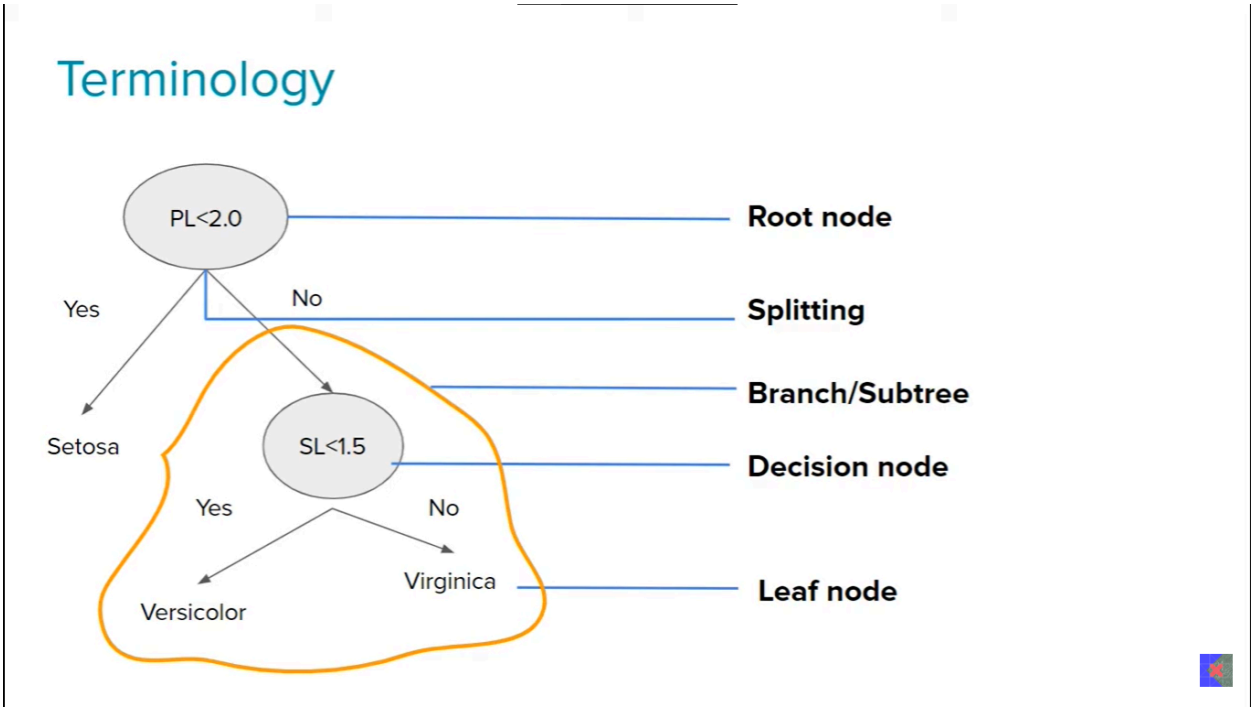
Conclusion

Programatically speaking, Decision trees are nothing but a giant structure of nested if-else condition

Mathematically speaking, Decision trees use hyperplanes which run parallel to any one of the axes to cut your coordinate system into hyper cuboids

Hyper cuboids : A line in 2D is a line but a Plane in 3D and a Hyperplane in ND. Cut/separate the Hyperplane using a line, each part is called hyper cuboid. For 3D, its cuboid.

4) Terminology



5) Questions :

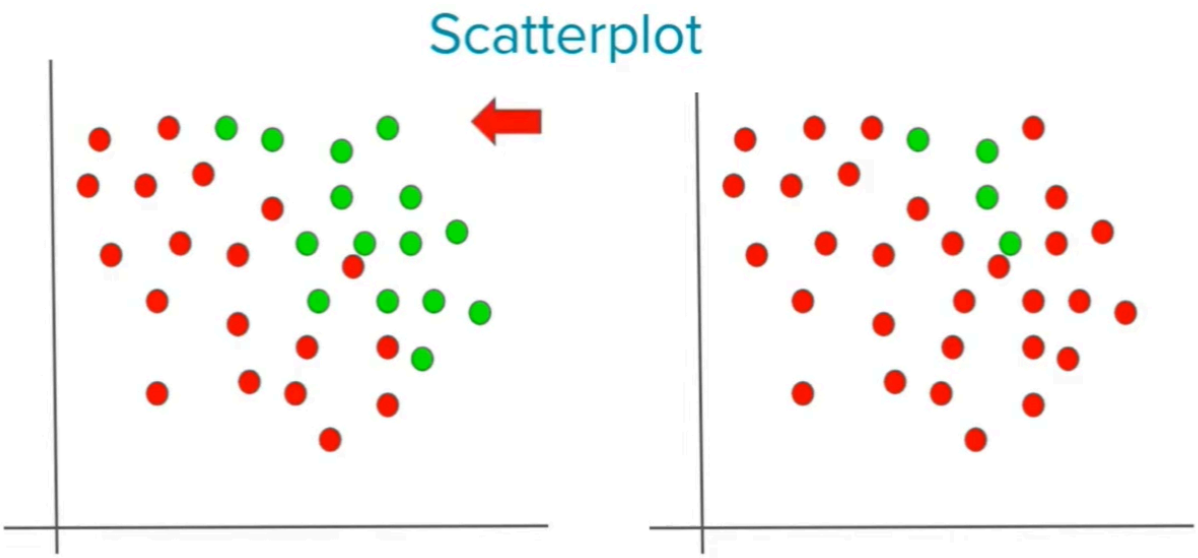
i) How to select the root node?

ii) After root node which node to select and so on?

iii) For numerical data, here we saw for petal_length < 2.0, it's 100% **Setosa**. But what if there was Versicolor whose petal_length also < 2.0? So how would we know which value to select to split the data to classify?

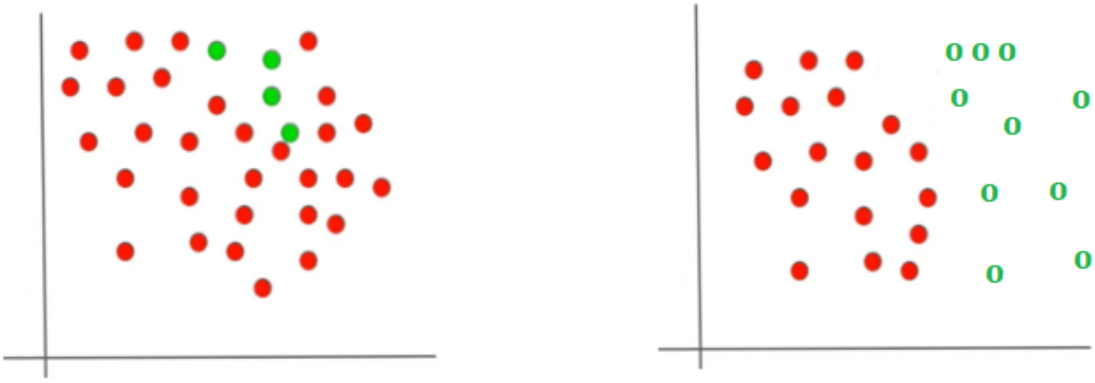
The answer of these questions we're going to describe at the next.

6) Entropy : (Certainty to classify separately)



The More the certainty/confidence to classify the data, the Less the entropy.

Entropy is the amount of **disorderness** in a system. We can easily see that in the first graph the disorderness is higher because we just can't easily classify the red and green points more than we can in the second graph.



Here in the 2nd graph, one side is purely red and the other side is purely green i.e. disorderness is very low / zero. So, we're **certain** with higher confidence that we can classify the second graph better than the 1st graph.

So, you can already guess that in a dataset comprising m columns, the first column which has the least entropy, is the first(Root/Decision node) node to select to split the dataset. Then continue the same with the rest of the dataset excluding that node. But actually we don't use entropy to select the best column to split the dataset, we use Information Gain which we'll learn soon.

7) How to calculate Entropy?

How to calculate Entropy?

The mathematical formula for entropy is:

$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

Where 'Pi' is simply the frequentist probability of an element/class 'i' in our data.

For e.g if our data has only 2 class labels **Yes** and **No.**

$$E(D) = -p_{yes} \log_2(p_{yes}) - p_{no} \log_2(p_{no})$$

Calculating entropy for a 3 class problem

Salary	Age	Purchase
20000	21	Yes
10000	45	No
60000	27	Yes
15000	31	No
30000	30	Maybe
12000	18	No
40000	40	Maybe
20000	20	Maybe

$H(d) = - P_y \log_2(P_y) - P_n \log_2(P_n) - P_m \log_2(P_m)$

$H(d) = -2/8 \log_2(2/8) - 3/8 \log_2(3/8) - 3/8 \log_2(3/8)$

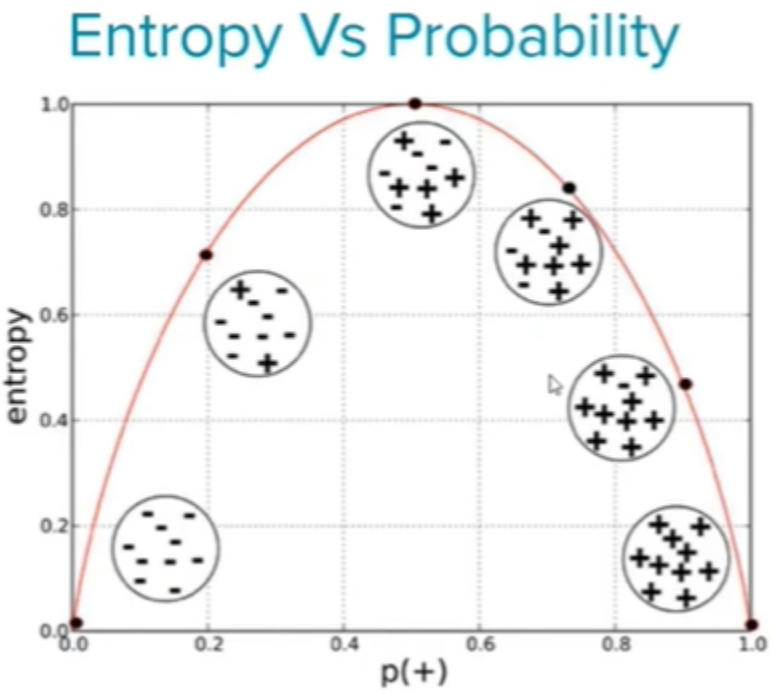
$H(d) = 1.56$

8) Observations

Observation

- More the uncertainty more is entropy
- For a 2 class problem the min entropy is 0 and the max is 1
- For more than 2 classes the min entropy is 0 but the max can be greater than 1
- Both $\log_2$ or $\log_e$ can be used to calculate entropy **because they both give larger value for bigger entropy and smaller value for smaller entropy which is our aim.**

9) **Entropy vs Probability(for 2 classes only, Positive & Negative, or Yes & No)**



When p(+) = 0, that means there are only negative values and when p(+) = 1, that means there are only positive values. In these both cases the entropy is obviously zero i.e. there is no entropy here. When p(+) = 0.5, $-p(+) * \log_2(p(+)) - p(-) * \log_2(p(-)) = 0.5 + 0.5 = 1$.

10) When the output column is a continuous variable i.e. Linear Regression using Decision Tree :

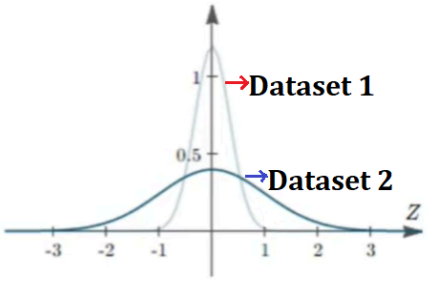
Entropy for continuous variables

Area	Built in	Price
1200	1999	3.5
1800	2011	5.6
1400	2000	7.3
...	...	...

Dataset 1

Area	Built in	Price
2200	1989	4.6
800	2018	6.5
1100	2005	12.8
...	...	...

Dataset 2



Quiz: Which of the above datasets have higher entropy?

Ans: Whichever is less peaked

Dataset 1, has less peakedness. So, it has less entropy.

11) Information Gain

Information Gain

Information Gain, is a metric used to train Decision Trees. Specifically, this metric measures the quality of a split.

The information gain is based on the decrease in entropy after a data-set is split on an attribute.Constructing a decision tree is all about finding attribute that returns the highest information gain

From google search labs:

Information gain is not the same as entropy, but it's closely related. Entropy measures the uncertainty or randomness in a dataset, while **information gain is the reduction in entropy achieved by splitting a dataset based on a particular feature/column**. And we want the column to split the

current dataset that can cause the **highest reduction in entropy**.

In simpler terms: Imagine you're trying to guess what color a box is by closing your eyes. If you have **no information (high entropy)**, it's very hard to guess. If you see the box is blue, the **uncertainty is reduced (low entropy), and the information gain is high** (you gained a lot of knowledge).

Information Gain = Entropy(Parent) - Entropy(Children)

Entropy(Children) is also known as Weighted Average.

12) Information Gain Calculation:

---

i) E(Parent) => Entropy of Parent. The Entropy of the entire dataset we're working on now.

Outlook	Temperature	Humidity	Windy	PlayTennis
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

Step 1:

Entropy of Parent

$E(P) = -p_y \log_2(p_y) - p_n \log_2(p_n)$

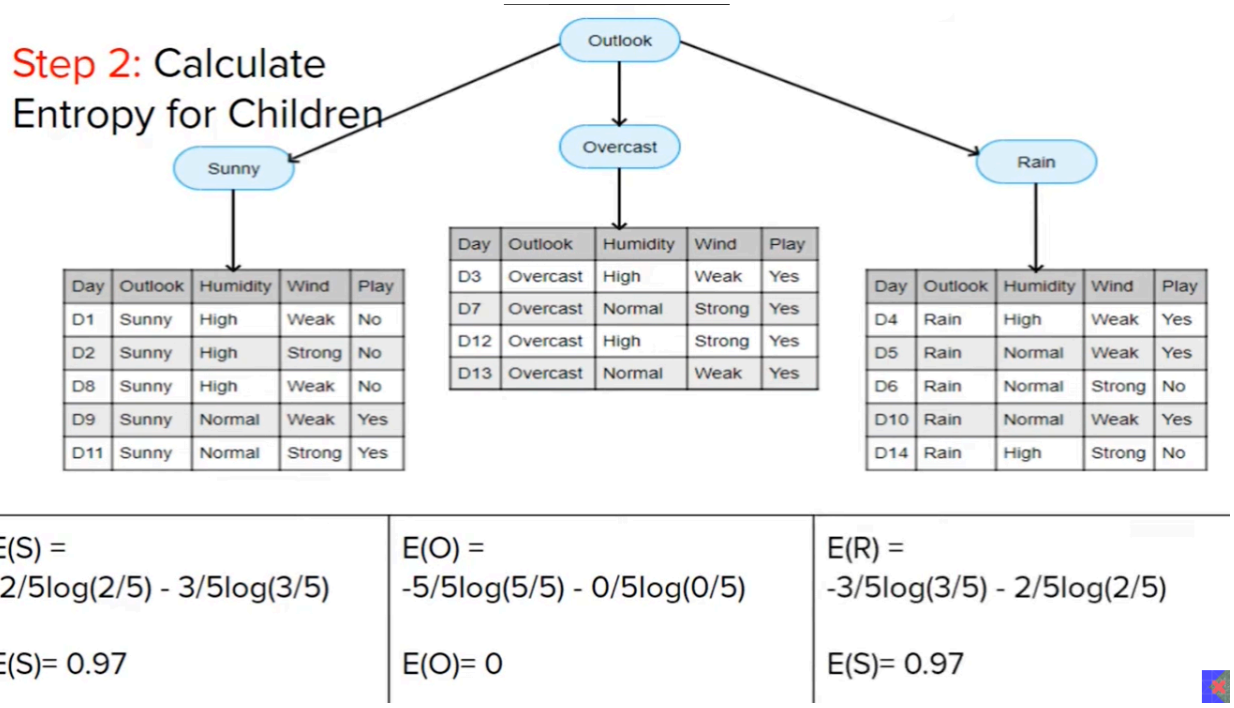
$= 9/14 \log_2(9/14) - 5/14/\log_2(5/14)$

$E(P) = \mathbf{0.94}$

ii) E(Children)

E(Childrens) => Entropy of Childrens separately. Whose childrens? The childrens(splitted dataset) we've after doing groupby

operation on a column. Here, we'll split the dataset based on the first column and calculate the entropy of the children dataset:



Day	Outlook	Humidity	Wind	Play
D1	Sunny	High	Weak	No
D2	Sunny	High	Strong	No
D8	Sunny	High	Weak	No
D9	Sunny	Normal	Weak	Yes
D11	Sunny	Normal	Strong	Yes

Day	Outlook	Humidity	Wind	Play
D3	Overcast	High	Weak	Yes
D7	Overcast	Normal	Strong	Yes
D12	Overcast	High	Strong	Yes
D13	Overcast	Normal	Weak	Yes

Day	Outlook	Humidity	Wind	Play
D4	Rain	High	Weak	Yes
D5	Rain	Normal	Weak	Yes
D6	Rain	Normal	Strong	No
D10	Rain	Normal	Weak	Yes
D14	Rain	High	Strong	No

E(S) = -2/5log(2/5) - 3/5log(3/5)	E(O) = -5/5log(5/5) - 0/5log(0/5)	E(R) = -3/5log(3/5) - 2/5log(2/5)
E(S)= 0.97	E(O)= 0	E(S)= 0.97

Weighted average => Calculate Total Weighted Entropy of childrens.

Weighted average = $P(Children_1) * E(Children_1) + P(Children_2) * E(Children_2) + P(Children_3) * E(Children_3) + ...$

$$P(Childen_1) = \frac{\text{Number of rows in Children}_1}{\text{Number of rows of Parent}}$$

So, Weighted average = (5/14) * 0.97 + (4/14) * 0 + (5/14) * 0.97
= 0.69

Weighted average = E(Children) = 0.69

Another point to notice is that E(Overcast) = 0 means it's completely pure by having only one type of output(Play = Yes) and thus no splitting can be made from this node. This node is called Leaf Node.

iii) Finally putting all the values to calculate Information Gain for the first column :

Step 4 : Calculate Information Gain

Information Gain = E(Parent) - E(Children)

IG = **0.94 - 0.69 = 0.25**

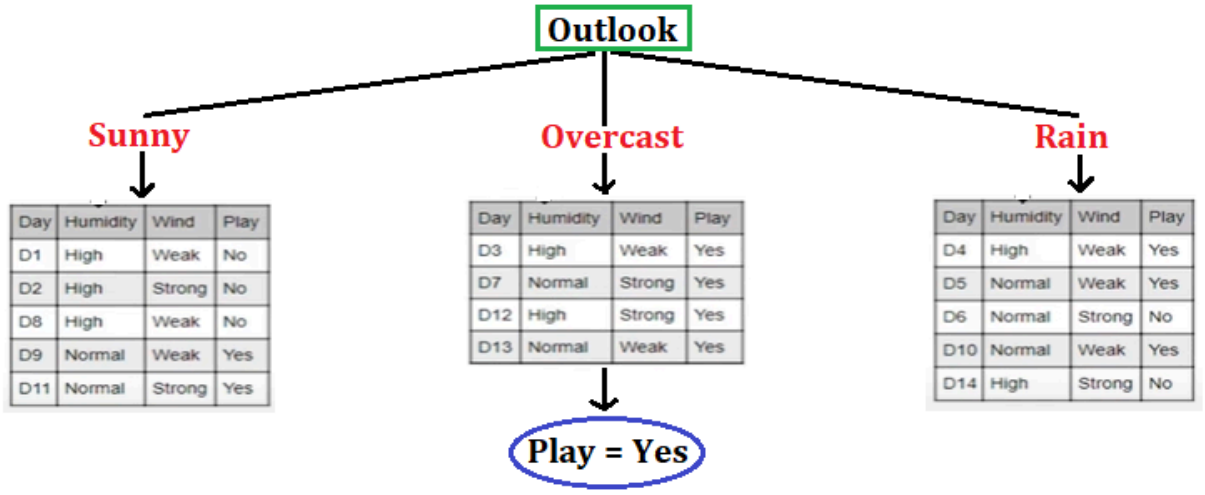
So the information gain(or the decrease in entropy/impurity) when you split this data on the basis of **Outlook** condition/column is **0.25.**

v) Calculate information gain for all the columns :

Step 5 : Calculate Information Gain for all the columns

Whichever column has the highest Information Gain(maximum decrease in entropy) the algorithm will select that column to split the data.

We just calculated information gain for the first column, Outlook. Now we have to calculate the information gain for the rest of the columns and then find the highest information gain and select the corresponding column to split the dataset to form the tree. Assume the **Outlook** column has the highest information gain. So My Tree is now :



Day	Humidity	Wind	Play
D1	High	Weak	No
D2	High	Strong	No
D8	High	Weak	No
D9	Normal	Weak	Yes
D11	Normal	Strong	Yes

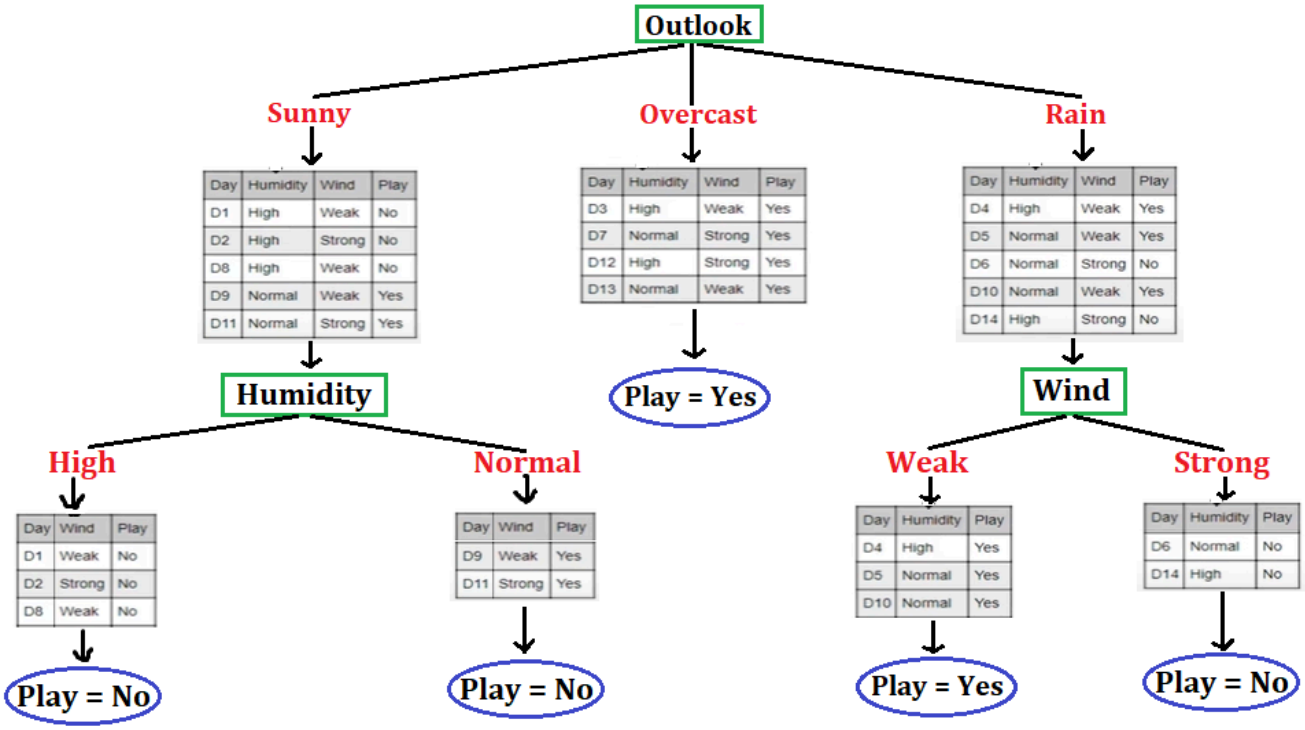
Day	Humidity	Wind	Play
D3	High	Weak	Yes
D7	Normal	Strong	Yes
D12	High	Strong	Yes
D13	Normal	Weak	Yes

Day	Humidity	Wind	Play
D4	High	Weak	Yes
D5	Normal	Weak	Yes
D6	Normal	Strong	No
D10	Normal	Weak	Yes
D14	High	Strong	No

If the dataset in 9th page is df. Then here **Sunny** denoting df.query("Outlook == 'Sunny'"). Similarly for Overcast and Rain. If you're thinking why from Sunny we need only this small filtered dataset having information only for the Sunny but not the entire dataset excluding obviously Outlook column; now if the user says the sky/Outlook is Sunny and later say Wind is Weak, this "Wind is Weak" is only for the information/rows which has Outlook == "Sunny" since user said at the first that sky/Outlook is Sunny. So we don't care about the rows where sky/Outlook is Overcast or Rain, right?

In **Overcast**, it doesn't matter what the value of Humidity or Wind is, the output is Yes which means if the sky/Outlook is Overcast, then you can play without worrying about anything else. So Overcast is playing as a Leaf Node here.

Now from Sunny and Rain, we've to recalculate all those 5 steps for the corresponding dataset until there is no more decision node to make decisions :



Now using the above Decision Tree (here it's used for classification) for any Outlook, Humidity and Wind, we can find out if we can play or not.

vi) Recursion :

Step 6 : Find Information Gain recursively

Decision tree then applies a recursive greedy search algorithm in top bottom fashion to find Information Gain at every level of the tree.

Once a leaf node is reached (Entropy = 0), no more splitting is done.

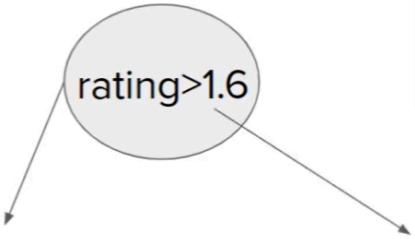
13) What if the feature/column/data is Numerical instead of categorical? (Below we took a dataset that has only 1 input numerical column) based on what we'll find/draw the decision tree.

Step 1:
Sort the data on the basis of numerical column

S No	User Rating	Downloaded
1	1.6	Yes
2	1.9	Yes
3	2.2	No
4	2.5	Yes
5	2.9	Yes
6	3.2	No
7	3.3	No
9	3.5	Yes
10	3.9	No
11	4.1	No
12	4.6	Yes
13	4.8	Yes

i)

Step 2:
Split the entire data on the basis of every value of user_rating



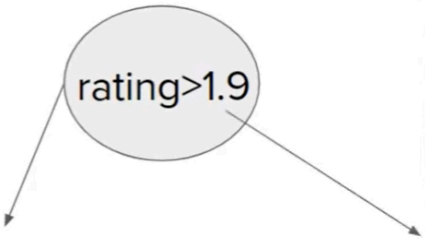
S No	User Rating	Downloaded
1	1.6	Yes

S No	User Rating	Downloaded
2	1.9	Yes
3	2.2	No
4	2.5	Yes
5	2.9	Yes
6	3.2	No
7	3.3	No
9	3.5	Yes
10	3.9	No
11	4.1	No
12	4.6	Yes
13	4.8	Yes

ii)

Step 2:

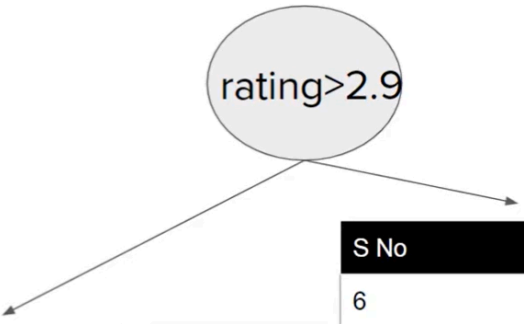
Split the entire data on the basis of every value of user_rating



S No	User Rating	Downloaded
1	1.6	Yes
2	1.9	Yes

S No	User Rating	Downloaded
3	2.2	No
4	2.5	Yes
5	2.9	Yes
6	3.2	No
7	3.3	No
9	3.5	Yes
10	3.9	No
11	4.1	No
12	4.6	Yes
13	4.8	Yes

iii)

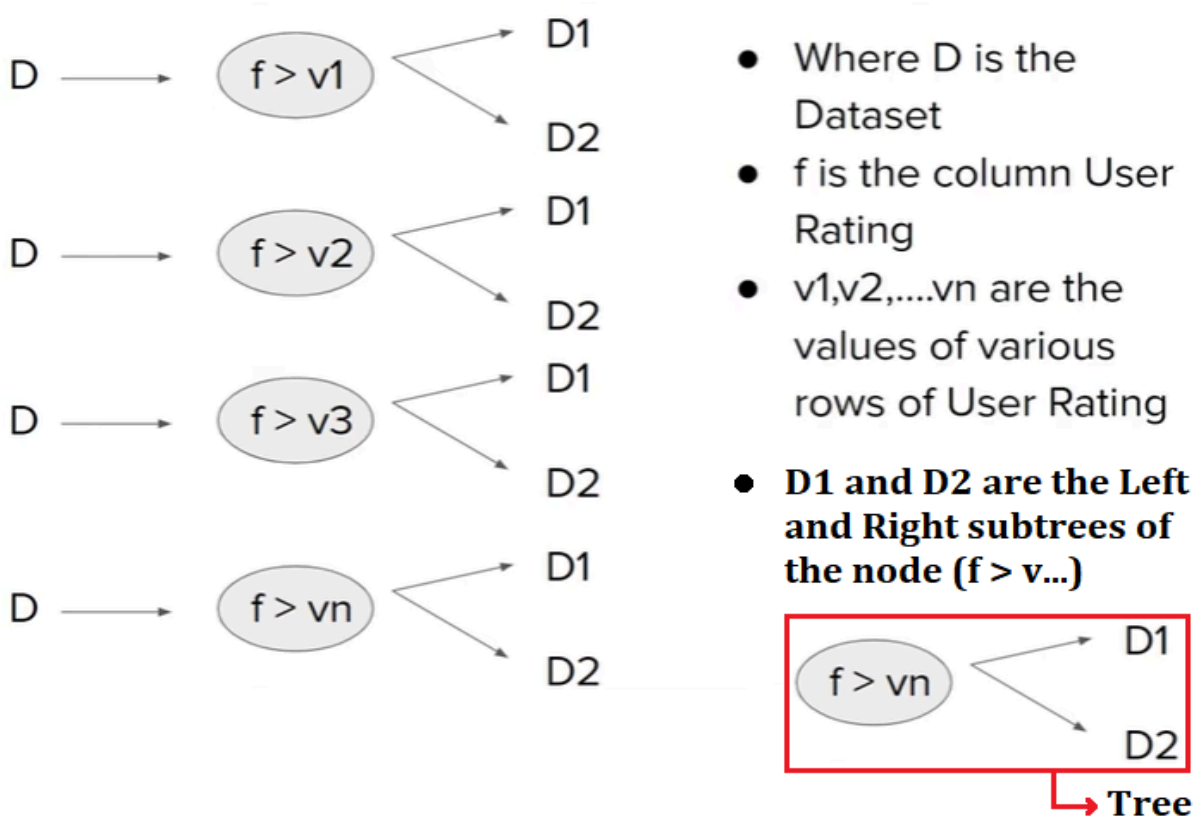


S No	User Rating	Downloaded
1	1.6	Yes
2	1.9	Yes
3	2.2	No
4	2.5	Yes
5	2.9	Yes

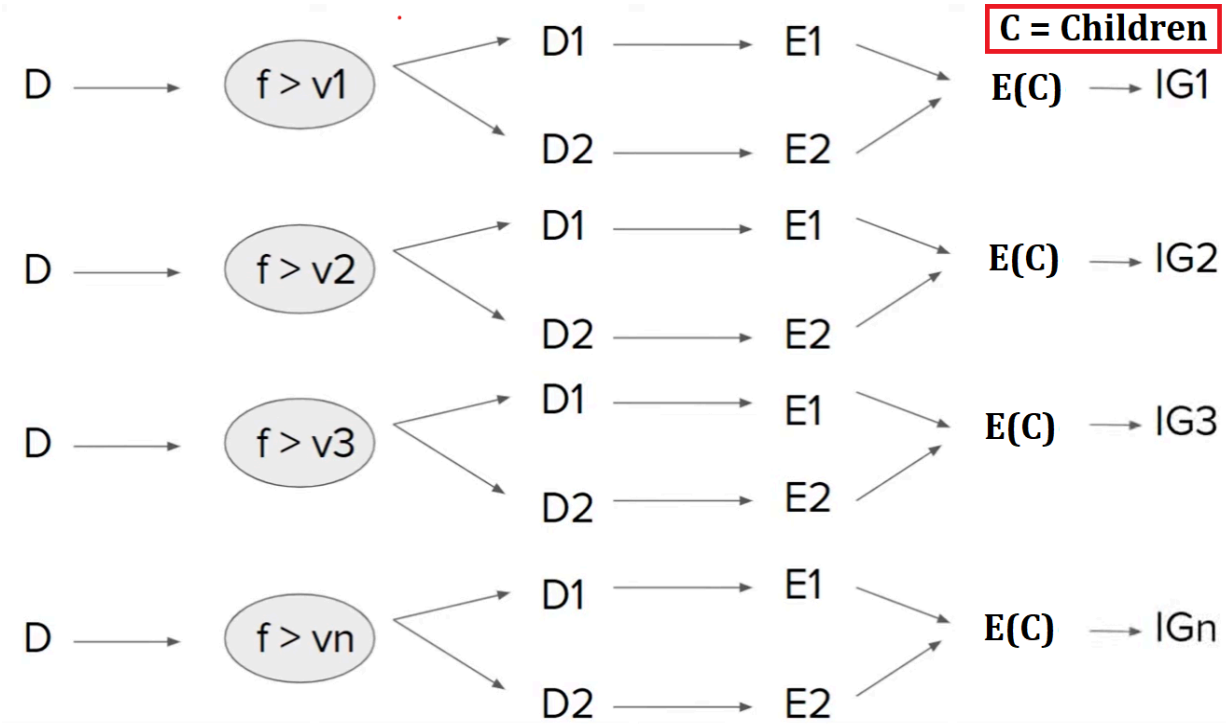
S No	User Rating	Downloaded
6	3.2	No
7	3.3	No
9	3.5	Yes
10	3.9	No
11	4.1	No
12	4.6	Yes
13	4.8	Yes

iv) Similarly, we've to do the same for the rest of the values of User Rating.

v) So at the end we'll get something like this :



vi) Calculate Information Gain for all the trees :



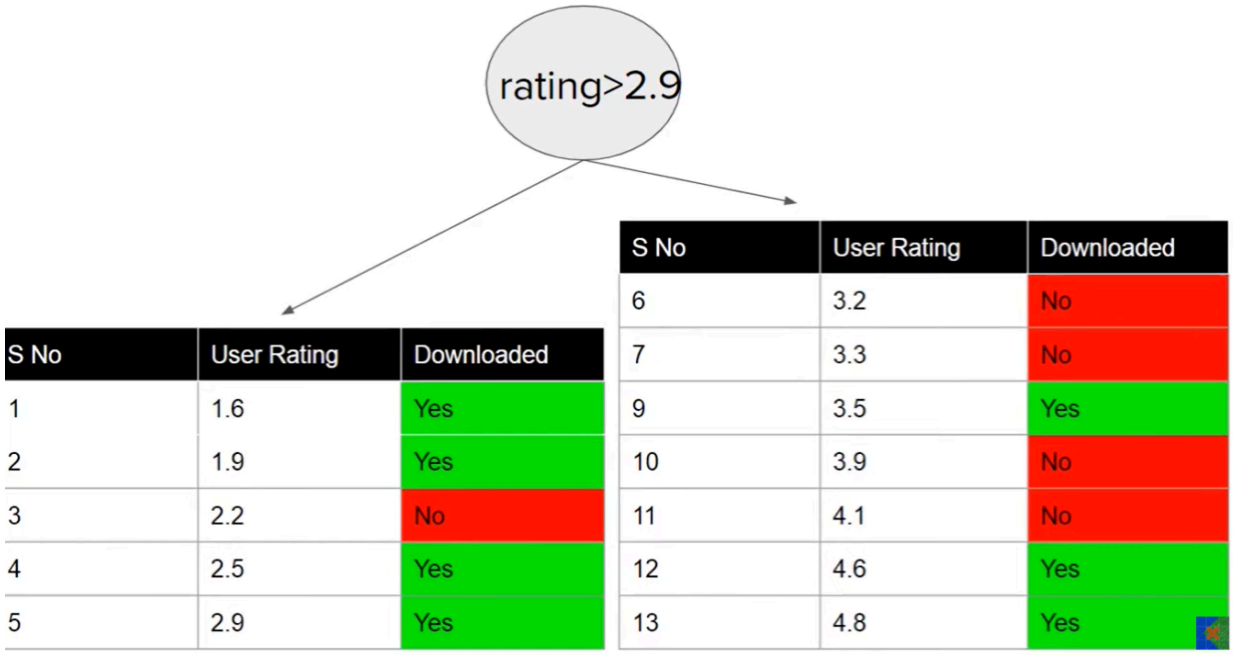
From **the Parent|Current_entire_dataset and (f > v1) comparison,** we've two subtrees|childrens, D1 and D2. From these 2 Childrens and Parent, we can calculate Information Gain.

From **the Parent|Current_entire_dataset and (f > v2) comparison,** we've two subtrees|childrens, D1 and D2. From these 2 Childrens and Parents, we can calculate Information Gain.

Similarly we calculate Information Gain for the other trees.

vii) Find the Maximum Information Gain and corresponding Comparison_Node(node having comparison, not a scalar value) :

From the above, we've [IG1, IG2, IG3, …, IGn]. Assume IG3 has the Maximum Information Gain. Now based on IG3, we'll split our dataset i.e. create the root node. IG3 belongs to **(rating > 2.9)** :



S No	User Rating	Downloaded
1	1.6	Yes
2	1.9	Yes
3	2.2	No
4	2.5	Yes
5	2.9	Yes

S No	User Rating	Downloaded
6	3.2	No
7	3.3	No
9	3.5	Yes
10	3.9	No
11	4.1	No
12	4.6	Yes
13	4.8	Yes

Now this is our current Decision Tree. We need to keep splitting i.e. repeating the above i-vi operations until there are only leaf nodes i.e. Output Node existing.

Gini Impurity

$$G = 1 - \left(\sum_{c=1}^{n} P_c^2\right)$$

c = the number of unique values in Output column.

For 2 unique values, Yes and No :

$$G = 1 - (P_y^2 + P_N^2)$$

Just like Entropy, Gini is also another measurement of Impurity|Certainty.

So we can also use “Gini” instead of “Entropy” inside Information Gain. Everything else is the same. Information Gain for Gini :

Information Gain = Gini(Parent) - Gini(Children)

Gini(Children) is also known as Weighted Average.

Gini(Children) = Weighted average = $P(Children_1) * G(Children_1) + P(Children_2) * G(Children_2) + P(Children_3) * G(Children_3) + \ldots..$

Since Gini is also a measurement of Impurity => The less the Gini, the more easily we can classify the data|column|feature, the more Information Gain we can have from that column|data|feature.

Salary	Age	Purchase
20000	21	Yes
10000	45	No
60000	27	Yes
15000	31	No
12000	18	No

$G = 1 - (Py^2 + Pn^2)$

$G = 1 - (4/25 + 9/25)$

$G = 0.48$

Salary	Age	Purchase
34000	31	No
15000	25	No
69000	57	Yes
25000	21	No
32000	28	No

$G = 1 - (Py^2 + Pn^2)$

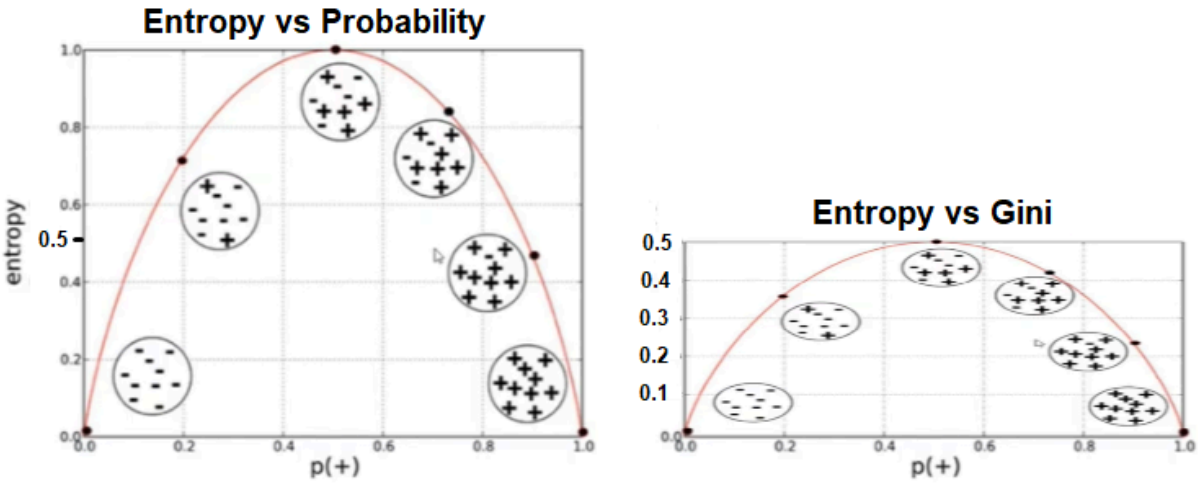
$G = 1 - (1/25 + 16/25)$

$G = 0.32$

Gini is less in the 2nd dataset. So we're certain here more that we can classify the dataset based on whether the user will Purchase or not. Also the Information Gain will be higher too than the 1st dataset.

Difference between Gini and Entropy

1) Square operation in Gini is computationally faster than Log operation in Entropy.
2)

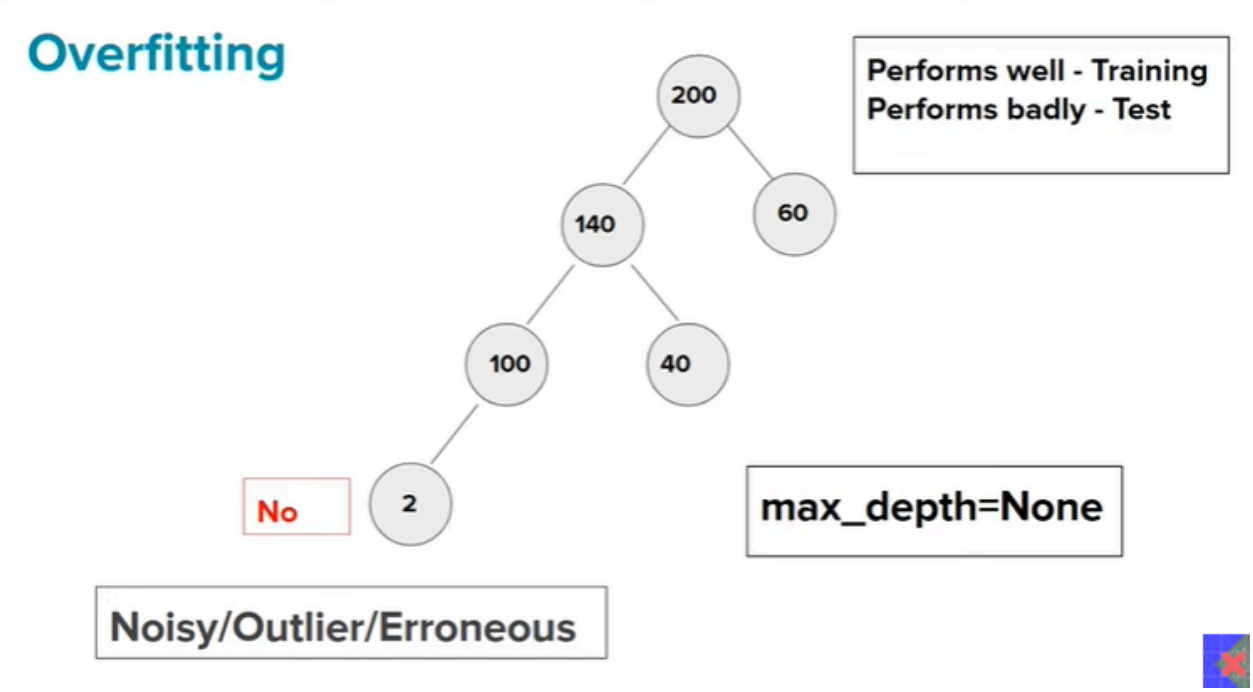


For P(Yes) = 0.5 and P(No) = 0.5 : Entropy returns 1 and Gini returns 0.5.

So when to use Gini and Entropy?

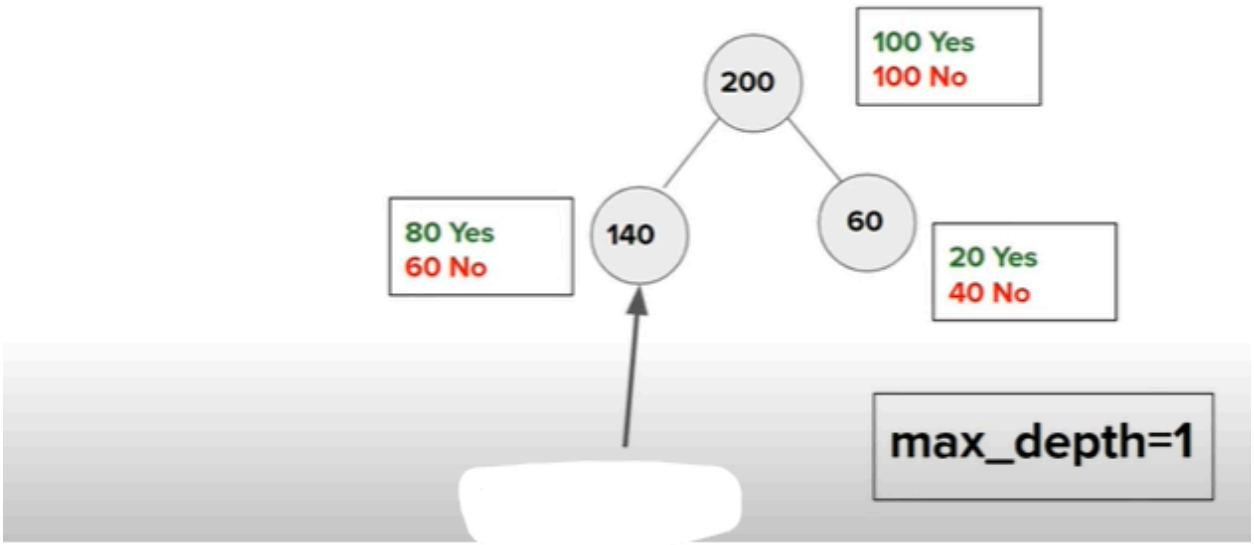
Use Gini most of the time since it is faster but in certain cases Entropy returns a more balanced tree and Gini causes Overfitting.

Overfitting and Underfitting

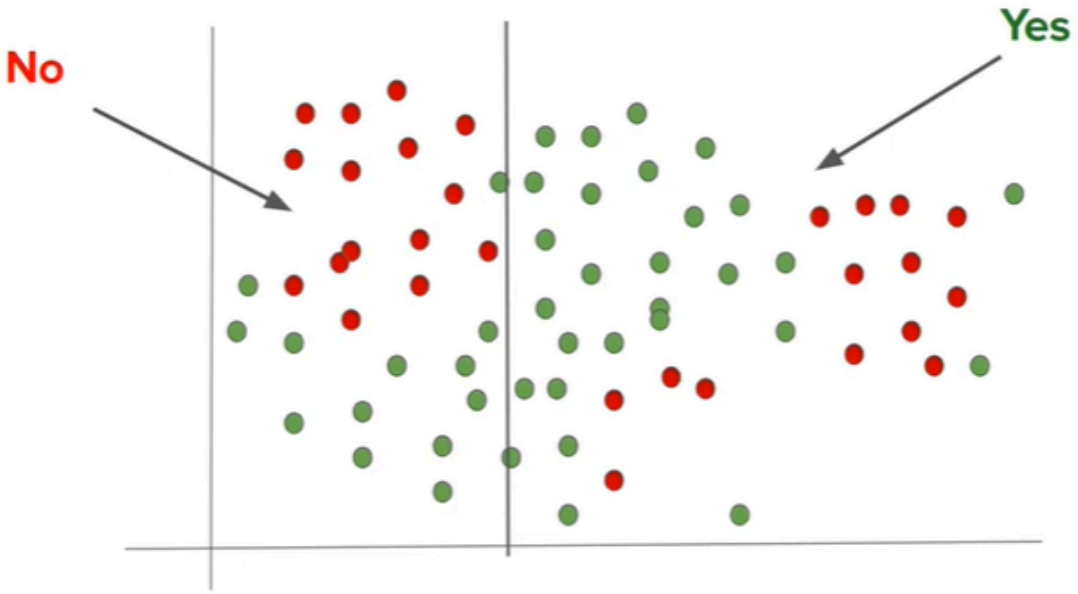


Imagine we have to search the number 2 in this tree all the way down and after doing so it turns out that it's actually an outlier or some noisy data. If your data has so many outliers or noisy data, then the decision tree will capture everything instead of learning the pattern and thus overfitting can easily be occurred by decision tree. So the more the depth of the decision tree, the more overfitting can occur.

Underfitting



Geometric Intuition of Underfitting



Here the maximum depth = 1 because we don't want anymore splitting i.e. now more depth level after first and just drew a line to classify the red and green points. Now we're assuming that any point left of that line is red and right after that line is green but it's not the actual scenario and underfitting causes this because of low max_depth. **So it's very crucial in the decision tree to set the optimal max_depth, so neither overfitting and underfitting are caused.**

In Decision Tree : The **higher** the depth, the more the information Decision Tree captures, the higher the chances of **overfitting**.
The **lower** the depth, the less the information Decision Tree captures, the higher the chances of **underfitting**.

Decision Tree important Hyperparameters

---

1) criterion = **{"gini", "entropy", "log_loss"},** default = **gini".**

2) splitter = **{"best", "random"},** default = **"best".**
   We quantify Information Gain for each feature/column and then split from the feature having Maximum Information Gain. This is "**best**". Instead of doing this, we can randomly select any feature/column from where we split. This is "**random**". "random" is another way to reduce overfitting.

   Chatgpt : Using `splitter='random'` can indeed help in reducing overfitting. By introducing randomness in the selection of splits, the model becomes less likely to capture noise in the training data, leading to better generalization on unseen data. This technique is particularly beneficial when used in ensemble methods like Random Forests, where multiple trees are built with different random splits, and their results are aggregated.

3) max_depth: **int,** default = **None**.
   Till what maximum depth the tree can split. If max_depth is lower, underfitting can cause and if a lot higher, overfitting can cause.

4) min_samples_split: **int or float,** default = **2.**
   A node(basically the subtree, page 18) can be further splitted if it has min_samples_split number of rows/samples left in that node.

   i) The lower the value of min_samples_split is, the more split we can make, the more information the Decision Tree can capture, the more chances of Overfitting and

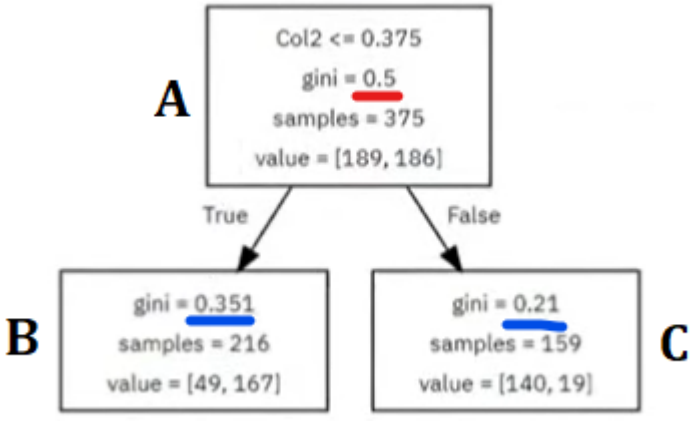
ii) The higher the value of min_samples_split is, the less split we can make, the less information the Decision Tree can capture, the more chances of Underfitting.

5) min_samples_leaf: **int or float,** default = **1.**
A Node can be splitted if after splitting its both left and right leaf node has min_samples_leaf number of samples left. The Overfitting and Underfitting issues are same as in the above step.

6) max_features: **int, float or {"sqrt", "log2"},** default = **None.**
When we split a dataset into more subset datasets, we use Information Gain from all the columns and select the maximum one. Instead of selecting Information Gain from all the columns, we can **select any n number of columns randomly** to calculate Information Gains and select the Maximum one. This is max_features. This is another way to reduce Overfitting by calculating Information Gain from Specific number of random columns, not all.

7) max_leaf_nodes: int, default = **None.**
Maximum number of leaf nodes to be used in the decision tree. The higher the number, the higher the chances of overfitting and the lower the number, the higher the chances of underfitting.

8) min_impurity_decrease: **float,** default = **0.0.**



Assume min_impurity_decrease = 0.01. Now node A can be splitted into B and C if

(B's impurity + C's impurity) - A's impurity
= (0.351 + 0.21) - 0.5
= 0.061 > 0.01. So, from A, B and C can be splitted.

But if min_impurity_decrease = 0.1, then it won't be possible to split.

9) **random_state** : *int, RandomState instance or None, default=**None.***

controls the randomness of the estimator. **The features are always randomly permuted at each split, even if "splitter" is set to "best"**. When `max_features < n_features`, the algorithm will select `max_features` at random at each split before finding the best split among them. But the best found split may vary across different runs, even if `max_features = n_features`. That's why if we want to have the same output(accuracy, prediction) at each run, we need to set random_state an integer value and the common value is 42 but you should see if increasing e.g. 50 gives you a better result or not.